

1. Estado Actual de la Arquitectura

El sistema de conversaciones del Mentor IA de VitaZen se compone actualmente de tres capas principales: el modelo de datos (Prisma), las APIs REST (Next.js route handlers), y un componente monolitico de interfaz (MentorChat.tsx). El pipeline de inteligencia se ha construido progresivamente a traves de las fases 2.x, anadiendo motores de contexto, razonamiento y personalidad que se ejecutan de forma no bloqueante antes de cada llamada a Groq. A continuacion se detalla la arquitectura existente con precision quirurgica.

1.1 Modelo de Datos (Prisma)

El sistema utiliza tres modelos principales para las conversaciones. AIThread representa una conversacion con campos basicos: id, userId, title (por defecto "Nueva conversacion"), archived (booleano), createdAt y updatedAt. AIMessage almacena cada mensaje individual con id, threadId, role (String sin enum: "user" o "assistant"), content y createdAt. AIUsage realiza un seguimiento del consumo diario con count, resetAt y una relacion uno-a-uno con User. El indice compuesto [userId, archived] en AIThread permite consultas eficientes para listar threads activos vs archivados, y [threadId, createdAt] en AIMessage permite recuperar mensajes en orden cronologico. No existen campos para favoritos, etiquetas, busqueda, ni eliminacion suave. El campo role es un String libre sin validacion a nivel de base de datos, lo que permite valores arbitrarios si el codigo no los restringe.

1.2 APIs Existentes

API	Metodo	Lineas	Funcion
POST /api/ai/chat	POST	382	Envio de mensaje + pipeline IA completo
GET /api/ai/threads	GET	183	Listar threads (archived filter)
POST /api/ai/threads	POST		Crear thread (max 20/100)
PATCH /api/ai/threads	PATCH		Renombrar/archivar thread
DELETE /api/ai/threads	DELETE		Eliminar thread (permanente)
GET /api/ai/threads/{id}/messages	GET	75	Listar mensajes (50/all)

La API de chat (route.ts, 382 lineas) implementa un pipeline robusto con 11 capas secuenciales: autenticacion y validacion, verificacion de limites diarios con lock advisory, adquisicion de lock por thread (pg_advisory_lock), carga de historial (10 FREE / 29 PREMIUM mensajes), construccion de contexto (buildMentorContext), Continuity Engine, Goals Engine, Reasoning Engine, Personality Engine, llamada a Groq (llama-3.3-70b-versatile con temp 0.5/0.8 y max_tokens 800/2048), y persistencia atomica de mensajes. Incluye mecanismos de rollback de creditos, generacion automatica de titulo en el primer intercambio, y extraccion fire-and-forget de objetivos. La API de threads soporta operaciones CRUD completas pero carece de busqueda, paginacion y soft delete. La API de mensajes solo soporta GET sin paginacion ni operaciones de edicion o eliminacion individual.

1.3 Componente MentorChat (UI)

El componente MentorChat.tsx es un monolito de 1.452 lineas que contiene toda la interfaz de conversacion: sidebar con lista de threads, area de chat, input de mensaje, menu contextual, modales de confirmacion y limites, drawer mobile, y logica de estado. Gestiona 20 hooks useState y 9 useRef internos, sin ningun hook custom ni store externo. Las funcionalidades implementadas incluyen: envio de mensajes con UI optimista, agrupacion por fecha (Hoy/Ayer/Esta semana/Este mes/Anterior), creacion/renombrado/archivado/eliminacion de threads con confirmacion, cambio de tab activas/archivadas, indicador de memoria contextual, limites Premium con blur de historial, chips de sugerencias estaticas (3 de 11), debounce de envio (1s), y deteccion de usuario offline. El componente carece de: streaming, renderizado markdown, input multilinea, copiar mensajes, editar mensajes, eliminar mensajes individuales, regenerar respuesta, buscar threads, favoritos/pins, exportar conversaciones, compartir, estadisticas visibles, atajos de teclado, y paginacion.

1.4 Pipeline de Inteligencia (Motores Existentes)

Motor	Archivo	Lineas	Latencia	DB Calls
buildMentorContext	mentor-context.ts	1.434	~200ms	~14
Contextual Continuity	continuity/engine.ts	587	~15ms	1
Goals Engine	goals/engine.ts	626	~10ms	1
Reasoning Engine	reasoning/engine.ts	790	<1ms	0
Personality Engine	personality/engine.ts	697	<1ms	0
Prompt Builder	groq.ts	106	<1ms	0
Limits	limits.ts	172	<1ms	1

Los siete motores de inteligencia se ejecutan de forma secuencial y no bloqueante. Si cualquier motor falla (envuelto en try/catch), el chat continua funcionando exactamente igual. El unico punto de fallo critico es la llamada a Groq, que consume el credito del usuario y dispara el rollback si falla. El Reasoning Engine y el Personality Engine operan exclusivamente sobre datos en memoria (<1ms cada uno), mientras que los motores de contexto (buildMentorContext, CCE, Goals) realizan consultas a la base de datos.

1.5 Limites FREE vs ELITE

Dimension	FREE	ELITE
Mensajes diarios	15	Ilimitados
Historial por thread	10 mensajes	29 mensajes
Threads activos maximo	20	100
Max tokens respuesta	800	2048
Temperature	0.5	0.8
Contexto del Mentor	Basico (check-in, rachas)	Completo (emocional, habitos, meditacion, journals, imperios, p
Continuidad contextual	No	Si (busqueda en hilos anteriores)
Objetivos activos	No	Si (extraccion + persistencia)
Memorias silenciosas	No	Si
Patrones detectados	No	Si
Cierre mensual	No	Si
Patron de personalidad	Simplificado	Completo con matices
Streaming	No	No
Markdown	No	No

2. Problemas Encontrados

2.1 Problemas Criticos (Bloqueantes de calidad)

Sin streaming: La respuesta completa tarda 2-5 segundos sin feedback visual. El usuario ve unicamente una animacion de tres puntos rebotando. En aplicaciones comparables (ChatGPT, Claude, Gemini), el streaming token-a-token es estandar y reduce drasticamente la percepcion de latencia. Groq soporta streaming nativo via su SDK, por lo que la implementacion no requiere cambios en el proveedor de IA, solo en el route handler y el componente frontend. Sin streaming, la experiencia percibida es significativamente inferior a la de cualquier competidor directo.

Sin renderizado markdown: Las respuestas del Mentor se muestran como texto plano con whitespace-pre-wrap. El sistema prompt instruye al modelo a evitar listas numeradas, pero el modelo genera naturalmente formato markdown (negritas, cursivas, listas con guiones, bloques de codigo) que se pierden completamente en la visualizacion. Esto hace que respuestas bien estructuradas por el modelo se

vean como bloques de texto denso y difícil de leer. Un parser markdown basado en react-markdown o similar resolvería esto sin impacto en rendimiento.

Componente monolito de 1.452 líneas: MentorChat.tsx contiene toda la lógica de UI en un solo archivo: sidebar, lista de threads, área de chat, input, menú contextual, modales, drawer mobile, y gestión de estado con 20 useState y 9 useRef. Esto hace que el archivo sea difícil de mantener, probar y evolucionar. Cualquier cambio requiere navegar un archivo masivo con alta cohesión accidental. La descomposición en componentes especializados (ThreadList, ChatMessages, MessageBubble, ChatInput, ContextMenu, etc.) es necesaria antes de añadir nuevas funcionalidades.

2.2 Problemas Médicos (UX deteriorada)

Sin copiar mensajes: No existe mecanismo para copiar el contenido de cualquier mensaje al portapapeles, ni en texto plano ni en markdown. Esta es una funcionalidad básica que todo chat de IA ofrece y cuya ausencia frustra al usuario cuando quiere guardar una respuesta especialmente valiosa del Mentor. La implementación es trivial (navigator.clipboard.writeText) y el impacto en UX es inmediato. Se recomienda añadir un botón de copia en cada mensaje del asistente con feedback visual (ícono check durante 2 segundos).

Sin regenerar respuesta: Cuando la respuesta del Mentor no es satisfactoria, el usuario no tiene forma de solicitar una regeneración. Debe escribir un nuevo mensaje pidiendo que lo intente de nuevo, consumiendo un crédito adicional e introduciendo ruido en el historial. La regeneración debería eliminar la última respuesta del asistente, reenviar el historial a Groq con la misma semilla de personalidad, y guardar la nueva respuesta. No consume crédito adicional si se implementa como un reintento con el mismo historial. El coste es una llamada adicional a Groq pero el valor percibido es muy alto.

Input de una sola línea: El campo de entrada usa input type="text", impidiendo al usuario escribir mensajes con múltiples líneas. Un mentor de desarrollo personal necesita recibir descripciones contextuales, reflexiones, y narrativas que naturalmente requieren varias líneas. El cambio a textarea es inmediato y sin riesgo de regresiones.

Sin búsqueda de conversaciones: Con 20 threads activos (FREE) y hasta 100 (ELITE), encontrar una conversación anterior requiere scroll manual. No existe filtro por texto, fecha ni etiqueta. A medida que el usuario acumula conversaciones, la usabilidad del sidebar degrada significativamente. Una búsqueda por título y contenido con debounce de 300ms es la mínima viable.

2.3 Problemas Técnicos (Deuda)

Logging inconsistente: Las rutas de threads y mensajes usan console.error en lugar del sistema serverLog del módulo de observabilidad. Esto significa que los errores en estas APIs son invisibles para el dashboard de monitorización. Además, la ruta de threads duplica el boilerplate de autenticación 4 veces (una por método HTTP), cuando podría extenderse a un wrapper o middleware.

Sin paginación en mensajes: La API de mensajes carga todos los mensajes del thread de una vez (FREE: 50, ELITE: todos). Un thread premium con cientos de mensajes podría ser lento de cargar y consumir mucha memoria en el cliente. La paginación con cursor (basada en createdAt) resolvería esto.

Eliminación permanente: DELETE en threads es destructivo e irreversible (CASCADE borra todos los mensajes). No existe papelera ni posibilidad de recuperación. Un usuario que elimina accidentalmente una conversación valiosa la pierde para siempre.

Código muerto: La función incrementAIUsage en limits.ts está marcada como @deprecated con cuerpo no-op. El campo streakReminders en NotificationPreference está marcado como DEPRECATED. El campo reflectionState en EmotionalDashboardState está marcado como DEPRECATED. Estos deberían limpiarse en una migración futura.

role como String sin enum: AIMessage.role es un String libre sin validación a nivel de Prisma. Cualquier valor arbitrario podría insertarse. Cambiar a un enum (user | assistant) proporcionaría validación a nivel de base de datos.

3. Arquitectura Definitiva Propuesta

La arquitectura propuesta se organiza en tres capas horizontales (datos, backend, frontend) con funcionalidades verticales que las atraviesan. Cada funcionalidad se ha evaluado contra cuatro criterios: utilidad real para el usuario, impacto diferencial frente a competidores, coste de mantenimiento, y coherencia con la filosofía VitaZen (equilibrio, honestidad, profundidad sin drama). Solo las funcionalidades que superan los cuatro criterios se proponen como imprescindibles.

3.1 Cambios al Modelo de Datos (Prisma)

Cambio	Modelo	Tipo	Justificacion
favorite	AIThread	Boolean @default(false)	Favoritos: acceso rapido a conversaciones importantes
pinnedAt	AIThread	DateTime?	Fijar conversaciones en la parte superior del sidebar
deletedAt	AIThread	DateTime?	Soft delete: eliminacion recuperable (papelera)
editedAt	AIMessage	DateTime?	Tracking de edicion de mensajes del usuario
parentMessageId	AIMessage	String?	Referencia al mensaje original antes de edicion
searchVector	AIThread	Unsupported("tsvector")	Indexacion full-text para busqueda rapida (PostgreSQL tsvector)

Los cambios al modelo de datos son minimos y backward-compatible. El campo deletedAt habilita el patron soft-delete sin romper la eliminacion permanente existente (se anade un filtro WHERE deletedAt IS NULL a las consultas). El campo searchVector permite busqueda full-text nativa de PostgreSQL sin necesidad de servicios externos como Algolia o Meilisearch. Los campos favorite y pinnedAt son columnas simples que no requieren migraciones complejas. El campo parentMessageId permite rastrear el historial de ediciones sin perder el contexto original.

3.2 Nuevas APIs Propuestas

API	Metodo	Descripcion	Complejidad
/api/ai/threads/search	GET	Busqueda full-text por titulo y contenido. Query param q con debounce 300ms.	Baja
/api/ai/threads/[id]/regenerate	POST	Regenerar ultima respuesta del asistente. Elimina ultima AIMessage, reenvia a Groq.	Media
/api/ai/threads/[id]/messages/[mid]	PATCH	Editar mensaje del usuario. Guarda version anterior via parentMessageId.	Media
/api/ai/threads/[id]/messages/[mid]	DELETE	Soft-delete de mensaje individual (marcado, no borrado).	Baja
/api/ai/threads/[id]/favorite	PATCH	Toggle favorite en thread.	Baja
/api/ai/threads/[id]/pin	PATCH	Toggle pin en thread.	Baja
/api/ai/threads/[id]/export	GET	Exportar conversacion como Markdown, TXT o PDF (query param format).	Media
/api/ai/threads/restore	POST	Restaurar thread desde papelera (clear deletedAt).	Baja
/api/ai/threads/empty-trash	DELETE	Eliminacion definitiva de threads con deletedAt != null.	Baja
/api/ai/threads/stats	GET	Estadisticas agregadas: total mensajes, tiempo, temas, frecuencia.	Baja

3.3 Descomposicion del Componente UI

La descomposicion propuesta divide MentorChat.tsx (1.452 lineas) en componentes especializados con responsabilidades unicas. Cada componente recibe props tipados y gestiona su propio estado cuando es necesario. La comunicacion entre componentes se realiza via callbacks lift-up hacia un contenedor MentorChatProvider que proporciona estado compartido via React Context.

Componente	Responsabilidad	Props clave	Estado
ThreadList	Lista de threads con agrupacion por temas	threads, activeId, onSelect, onArchive, onDelete, onRegenerate, onSearch	selectedThread, searchQuery
ThreadListItem	Fila individual de la lista	thread, isActive, onClick, onContextMenu, onPin, onFavorite	thread, isPinned, isFavorite
ChatMessages	Contenedor de mensajes con scroll automatico	messages, isSending, onCopy, onRegenerate, onEdit, onMore	scrollRef, isLoadingMore
MessageBubble	Mensaje individual (user o assistant)	message, onCopy, onRegenerate, onEdit, onPin	copied, isEditing

ChatInput	Campo de entrada con textarea multi-linea	onSearch, disabled, placeholder, remaining, isFocused	
ContextMenu	Menu de acciones sobre thread/item	position, items[], onClose	open, position
SearchBar	Busqueda de conversaciones	onSearch, placeholder	query, debouncedQuery
StatsPanel	Estadísticas de uso del Mentor	stats	none
ExportModal	Modal de exportacion de conversaciones	thread, formats[], onClose	selectedFormat

La descomposicion permite anadir nuevas funcionalidades (como exportacion, estadísticas o busqueda avanzada) como componentes independientes sin tocar el resto de la interfaz. Cada componente es testeable unitariamente y reutilizable. El contenedor MentorChatProvider centraliza el estado de threads, mensajes y sesion, eliminando la necesidad de prop-drilling excesivo.

4. Analisis Detallado por Funcionalidad

4.1 Busqueda Global de Conversaciones

Utilidad: Con 20-100 threads activos, la busqueda se vuelve imprescindible. Un usuario que busca una conversacion sobre "estres laboral" o "rutina de manana" no puede hacerlo actualmente. La busqueda es la funcionalidad con mayor impacto inmediato en la usabilidad diaria del sistema.

Arquitectura: Se anade una columna searchVector de tipo tsvector en AIThread. Un trigger de PostgreSQL (o un hook Prisma post-create/post-update) actualiza automaticamente el vector con el titulo y el contenido de los ultimos N mensajes del thread. La busqueda se realiza via Prisma raw query con to_tsquery y ranking por ts_rank. La API acepta un query param "q" y devuelve threads ordenados por relevancia. El frontend implementa un SearchBar con debounce de 300ms que filtra la lista en tiempo real.

Rendimiento: PostgreSQL tsvector con GIN index permite busquedas full-text en <5ms sobre miles de threads. El trigger se ejecuta solo en create/update, no en cada read. No requiere servicios externos ni embedding models. La actualizacion del vector se limita a los ultimos 10 mensajes para mantener el tamano del vector acotado.

Privacidad: La busqueda se realiza sobre los propios datos del usuario (filtro WHERE userId). No se comparte informacion entre usuarios. El tsvector se almacena en la misma tabla, no en un indice externo.

4.2 Favoritos y Fijados

Utilidad: Los favoritos permiten al usuario marcar conversaciones importantes para acceso rapido. Los fijados (pinned) mantienen conversaciones en la parte superior del sidebar independientemente de su fecha. Estas funcionalidades transforman el sidebar de una lista cronologica simple en un sistema de organizacion personal.

Arquitectura: Se anade favorite (Boolean) y pinnedAt (DateTime?) a AIThread. La API PATCH /api/ai/threads/[id]/favorite hace toggle del booleano. La API PATCH /api/ai/threads/[id]/pin establece o limpia pinnedAt. El listado de threads ordena primero los fijados (ORDER BY pinnedAt DESC NULLS LAST), luego por updatedAt DESC. Un icono de estrella distingue los favoritos en la lista.

Coste: Dos campos booleano/timestamp en Prisma, dos endpoints PATCH triviales, y logica de ordenamiento en la query existente. Impacto en rendimiento: nulo (el ORDER BY ya existe, solo se anade una condicion NULLS LAST).

4.3 Archivado Mejorado

Utilidad: El archivado ya existe (campo archived en AIThread + tab en UI). La mejora propuesta consiste en anadir la capacidad de archivar desde el menu contextual de forma mas accesible, y en mostrar threads archivados con fecha de archivado en lugar de fecha de creacion.

Estado: Funcionalidad existente. Solo requiere mejoras menores de UX. No se propone ningun cambio al modelo de datos ni a las APIs para esta funcionalidad.

4.4 Eliminacion con Papelera

Utilidad: La eliminacion permanente sin confirmacion ni recuperacion es un riesgo elevado. Un usuario que elimina accidentalmente una conversacion con reflexiones valiosas la pierde para siempre. La papelera (soft delete) es un estandar de la industria que proporciona una red de seguridad sin anadir complejidad significativa.

Arquitectura: Se anade `deletedAt` (`DateTime?`) a `AIThread`. El `DELETE` existente se convierte en soft delete (establece `deletedAt = now()`). Se anade un filtro `WHERE deletedAt IS NULL` a todas las consultas de listado. Se anade `/api/ai/threads/restore` (`POST`) para restaurar y `/api/ai/threads/empty-trash` (`DELETE`) para eliminacion definitiva. La UI muestra una pestana "Papelera" junto a "Archivadas" con threads eliminados y opcion de "Vaciar papelera". Los threads en papelera se eliminan automaticamente tras 30 dias via un cron job.

Rendimiento: El filtro `deletedAt IS NULL` se absorbe por el indice existente [`userId`, `archived`] anadiendo `deletedAt` al indice compuesto. El cron de limpieza es una query `DELETE` simple que se ejecuta una vez al dia fuera de horas punta.

4.5 Exportacion de Conversaciones

Utilidad: Los usuarios invierten tiempo y reflexion en sus conversaciones con el Mentor. Poder exportarlas como Markdown o TXT permite conservarlas fuera de la aplicacion, usarlas en otras herramientas, o simplemente tener un respaldo local. La exportacion a PDF tiene menor prioridad dado el coste de generacion.

Arquitectura: API `GET /api/ai/threads/[id]/export?format=markdown|txt|pdf`. El servidor carga todos los mensajes del thread, los formatea segun el formato solicitado, y devuelve el archivo como blob. Markdown: formato nativo con cabecera de titulo y fecha. TXT: texto plano sin formato. PDF: generacion server-side con ReportLab o similar. El frontend muestra un modal con tres opciones de formato y un boton de descarga.

Coste IA: Cero. La exportacion es formateo puro, no genera llamadas a Groq. El coste computacional es el de una query DB mas el formateo, ambos despreciables.

Privacidad: Solo el propietario del thread puede exportarlo (verificacion de `userId`). Los archivos se generan on-demand y no se almacenan en el servidor.

4.6 Compartir Conversaciones

Utilidad: Baja. Compartir conversaciones de coaching personal plantea serios problemas de privacidad (contienen reflexiones intimas, datos emocionales, objetivos personales). La exportacion individual cubre la necesidad de conservar y reutilizar contenido. Recomendacion: descartar en esta fase. Si se implementara en el futuro, deberia ser opt-in por mensaje (no por conversacion completa) y con ofuscacion de datos sensibles.

Veredicto: DESCARTAR. El riesgo de privacidad supera el beneficio. La exportacion cubre la necesidad de conservacion.

4.7 Copiar Respuestas

Utilidad: Muy alta. Es la funcionalidad mas solicitada y mas facil de implementar. Los usuarios quieren copiar respuestas del Mentor para usarlas como referencia, compartirlas parcialmente, o guardarlas en notas externas. Sin copiar, la unica forma de extraer contenido es seleccionar manualmente el texto, lo cual es tedioso en movil.

Arquitectura: Se anade un boton de copia (icono clipboard) en cada `MessageBubble` del asistente. Al hacer click, se ejecuta `navigator.clipboard.writeText(content)`. Se muestra feedback visual (icono check durante 2 segundos). Opcion: doble click para copiar automaticamente. No requiere cambios en backend ni en Prisma. Implementacion puramente frontend.

Rendimiento: Cero impacto. Es una operacion del navegador que no genera ninguna llamada al servidor. El icono de copia se renderiza como SVG inline sin dependencias externas.

4.8 Regenerar Respuesta

Utilidad: Alta. Cuando el Mentor genera una respuesta inadecuada (demasiado generica, mal enfocada, o simplemente no util), el usuario necesita poder solicitar una regeneracion sin escribir un nuevo mensaje. Actualmente debe escribir algo como "intenta de nuevo" o "no me sirve, dame otra respuesta", lo cual consume un credito y anade ruido al historial.

Arquitectura: API POST /api/ai/threads/[id]/regenerate. El endpoint: (1) verifica que el ultimo mensaje sea del asistente, (2) lo elimina, (3) reconstruye el historial completo sin la ultima respuesta, (4) reenvia a Groq con el mismo prompt del sistema, (5) guarda la nueva respuesta. El costo es una llamada adicional a Groq pero NO consume un credito diario (la regeneracion es un reintento, no un mensaje nuevo del usuario). El boton de regenerar aparece solo en el ultimo mensaje del asistente. Se deshabilita durante el envio para evitar dobles clics.

Coste IA: Una llamada a Groq por regeneracion. A \$0.05/1M tokens de entrada y \$0.08/1M de salida, una regeneracion tipica cuesta ~\$0.001. Con el limitador de tasa (1 debounce de envio), no hay riesgo de abuso. Se recomienda limitar a 3 regeneraciones por thread para evitar bucles infinitos.

4.9 Editar Mensajes del Usuario

Utilidad: Media. Permite al usuario corregir errores tipograficos o ampliar un mensaje despues de enviarlo. Sin embargo, editar un mensaje que ya fue procesado por el Mentor crea un problema de coherencia contextual: la respuesta del asistente fue generada basandose en el mensaje original, no en el editado. La solucion es editar el mensaje Y regenerar la respuesta posterior.

Arquitectura: Se anade editedAt (DateTime?) y parentMessageId (String?) a AIMessage. Al editar, se guarda el mensaje original como registro (parentMessageId apunta al id del mensaje antes de la edicion), se actualiza el contenido, y se eliminan todos los mensajes del asistente posteriores al mensaje editado. El usuario debe confirmar que la edicion eliminara las respuestas posteriores. Se ejecuta automaticamente una regeneracion de la respuesta inmediatamente posterior al mensaje editado.

Riesgo: Eliminar respuestas del asistente puede ser frustrante si el usuario no entiende las consecuencias. Se recomienda un dialogo de confirmacion claro: "Al editar este mensaje se eliminaran 3 respuestas del Mentor y se generara una nueva. Continuar?"

4.10 Estadisticas de Conversacion

Utilidad: Media-alta. Ver cuantos mensajes se han enviado, cuanto tiempo se ha invertido, que temas se tratan mas, y como evoluciona el uso del Mentor a lo largo del tiempo proporciona al usuario una vision de su progreso que refuerza el compromiso. Sin embargo, las estadisticas detalladas (temas, evolucion) requieren procesamiento adicional que puede no justificar el coste en esta fase.

Arquitectura: Estadisticas basicas (totales) calculadas en el cliente a partir de datos ya cargados: total de mensajes, threads activos, y frecuencia de uso (ultima semana vs semana anterior). Estadisticas avanzadas (temas, evolucion temporal, objetivos completados) requeririan una API dedicada con queries agregadas. Recomendacion FASE 3.1: estadisticas basicas en el sidebar. Estadisticas avanzadas para FASE 3.2.

4.11-4.20 Integraciones con el Ecosistema VitaZen

Las integraciones con los motores existentes (Goals, CCE, Emotional Understanding, habitos, diario, nutricion, finanzas, imperios) ya estan implementadas y operativas a traves del pipeline de buildMentorContext y los motores especializados. No se proponen cambios a estas integraciones en FASE 3.1. Sin embargo, la arquitectura propuesta habilita mejoras futuras de integracion que se describen a continuacion.

Integracion	Estado Actual	Mejora Futura (FASE 3.2+)
-------------	---------------	---------------------------

Goals Engine	Funcional: extraccion + inyeccion de datos	Mostrar objetivos activos en sidebar de conversacion
Contextual Continuity	Funcional: busqueda en hilos anteriores	Resaltar conexiones en la UI ("Continuacion de...")
Emotional Understanding	Funcional: inyeccion en prompt	Mostrar estado emocional detectado en el header del chat
Habitos	Funcional: datos en buildMentorContext	Proponer acciones basadas en habitos activos en la conversacion
Diario (Journal)	Funcional: journals en contexto	Referenciar entradas del diario cuando el usuario menciona temas relacionados
Nutricion	Funcional: datos en contexto PREMIUM	Conectar conversaciones de alimentacion con datos reales
Finanzas	Funcional: datos en contexto PREMIUM	Ofrecer perspectivas financieras basadas en datos reales del usuario
Imperios	Funcional: progreso en contexto	Mostrar progreso de imperios relevantes en el sidebar
Pattern Detection	Funcional: patrones en contexto PREMIUM	Alertar sobre patrones detectados cuando son relevantes
Life Stages	Funcional: etapa en contexto PREMIUM	Ajustar profundidad y tono segun la etapa actual del usuario

La clave de las mejoras futuras es que los datos ya existen en el contexto del Mentor (inyectados en el prompt del sistema). Lo que falta es hacer visibles estos datos en la interfaz. Por ejemplo, cuando el Goals Engine detecta un objetivo de "caminar 10.000 pasos", el sidebar podria mostrar ese objetivo como contexto activo de la conversacion. Cuando el Emotional State Engine detecta "sobrecargado", el chat podria mostrar un indicador sutil de "estado: estresado" que ayude al usuario a entender por que el Mentor responde con mas empatia. Estas mejoras son de presentacion, no de procesamiento.

5. Diferencias FREE vs ELITE Propuestas

La diferenciacion entre planes debe sentirse como calidad, no como longitud ni como funciones bloqueadas artificialmente. Un usuario FREE debe tener una experiencia completa y funcional; un usuario ELITE debe sentir que su Mentor realmente le conoce, conecta puntos entre conversaciones, y adapta su profundidad con precision quirurgica.

Funcionalidad	FREE	ELITE
Busqueda	Busqueda por titulo	Busqueda por titulo + contenido full-text
Historial por thread	10 mensajes	29 mensajes (ya implementado)
Regenerar respuesta	1 regeneracion por conversacion	3 regeneraciones por conversacion
Editar mensajes	No disponible	Disponible (con regeneracion automatica)
Exportar	Solo TXT	Markdown + TXT + PDF
Estadisticas	Basicas (total mensajes)	Avanzadas (temas, evolucion, frecuencia)
Favoritos	Hasta 3 favoritos	Ilimitados
Fijados	No disponible	Disponible
Papelera	7 dias de recuperacion	30 dias de recuperacion
Sugerencias	3 chips estaticos	Contextuales basadas en Goals + emotional state
Streaming	Si (implementacion base)	Si (con indicador de tokens)
Markdown	Basico (negritas, listas)	Completo (tablas, codigo, cabeceras)
Integraciones visibles	Estado emocional en header	Estado emocional + objetivos + patrones + etapa de vida

6. Riesgos

Riesgo	Severidad	Mitigacion
Streaming rompe el patron non-blocking	Alta	Implementar streaming como capa separada. Si falla, fallback a respuesta completa.
Descomposicion de MentorChat introduce regresiones	Alta	Descomponer incrementalmente. Cada componente en su propio PR. Tests de regresion.
Busqueda full-text requiere migracion Prisma	Media	Crear columna con @default(null). Poblar con trigger post-deploy. Sin downtime.
Regeneracion consume recursos de Groq	Media	Limitar a 3 por conversacion. No consume credito del usuario pero cuenta para rate limit.

Edicion de mensajes confunde al usuario	Media	Dialogo de confirmacion claro. Preview del impacto antes de confirmar.
tsvector en PostgreSQL requiere extension	Baja	Prisma soporta @default(dbgenerated("to_tsvector(...)")) para columnas computed. Alto
Exportacion a PDF es costosa en servidor	Baja	Limitar a threads < 100 mensajes. Generacion lazy (on-demand). Cache de 5 minutos.

7. Coste Computacional

Todas las funcionalidades propuestas estan disenadas para tener un impacto minimo en el coste computacional. La busqueda full-text con tsvector a ade <5ms a las queries de listado. Los favoritos, fijados y papelera son campos booleanos/timestamps en consultas ya existentes. La exportacion es on-demand sin almacenamiento. El streaming no a ade coste de procesamiento, solo cambia el patron de respuesta. La unica funcionalidad con coste adicional significativo es la regeneracion de respuestas, que a ade una llamada a Groq (~\$0.001 por regeneracion).

Funcionalidad	Coste DB adicional	Coste IA adicional	Coste frontend
Busqueda full-text	<5ms por query (GIN index)	\$0	Debounce 300ms
Favoritos/Pin	0 (mismo query, distinto ORDER BY)	\$0	1 icono SVG
Papelera	1 campo + filtro WHERE	\$0	1 pestana + 1 boton
Exportacion TXT/MD	1 query mas (todos los mensajes)	\$0	Modal + blob download
Exportacion PDF	1 query + generacion ~500ms	\$0	Idem + cache
Copiar mensaje	\$0	\$0	1 boton + clipboard API
Regenerar respuesta	1 DELETE + 1 query + 1 Groq call	~\$0.001	1 boton + estado
Editar mensaje	1 UPDATE + regeneracion posterior	~\$0.001	Textarea inline + dialogo
Estadisticas basicas	0 (calculadas en cliente)	\$0	Contador en sidebar
Markdown renderizado	0	0	react-markdown ~15KB gzip

8. Coste Economico Estimado

El coste economico de implementar la FASE 3.1 se compone de tres recursos principales: desarrollo (horas de ingenieria), infraestructura (recursos de Groq), y mantenimiento a largo plazo. Las estimaciones asumen un desarrollador senior trabajando a ritmo sostenible.

Item	Horas estimadas	Coste relativo
Descomposicion de MentorChat	16-20h	Alto (refactor sin funcionalidad nueva)
Streaming (backend + frontend)	12-16h	Alto (cambio arquitectural)
Renderizado Markdown	4-6h	Bajo (libreria existente)
Busqueda full-text	6-8h	Medio (migracion + API + UI)
Copiar mensajes	1-2h	Minimo
Regenerar respuesta	4-6h	Medio (API + UI + logica de creditos)
Editar mensajes	6-8h	Medio (API + UI + parentMessageId + regeneracion)
Favoritos + Pin	3-4h	Bajo
Papelera	4-6h	Medio (soft delete + cron + UI)
Exportacion	6-8h	Medio (3 formatos + modal + blob)
Estadisticas basicas	2-3h	Bajo
Multiline input	0.5h	Minimo
Estadisticas avanzadas (FASE 3.2+)	8-12h	Alto
TOTAL FASE 3.1	73-99h	

El coste recurrente es despreciable: la busqueda full-text usa indices GIN de PostgreSQL que ya estan optimizados, la papelera usa un cron diario con una query DELETE simple, y las estadisticas basicas se calculan en el cliente. El unico coste recurrente de IA es la regeneracion de respuestas (~\$0.001 por regeneracion), que se limita a 3 por conversacion. A un ritmo de 100 usuarios activos con 1 regeneracion diaria cada uno, el coste mensual seria de ~\$3.

9. Complejidad y Dependencias

Funcionalidad	Complejidad	Dependencias nuevas	Riesgo de regresion
Streaming	Alta	groq SDK stream API	Medio
Markdown	Baja	react-markdown ~15KB	Bajo
Multiline input	Minima	Ninguna	Nulo
Copiar mensajes	Minima	Ninguna	Nulo
Busqueda	Media	tsvector GIN index	Bajo
Favoritos/Pin	Baja	Ninguna	Nulo
Papelera	Media	Cron job	Bajo
Regenerar	Media	Ninguna	Medio
Editar mensajes	Alta	parentMessageId en schema	Alto
Exportacion TXT/MD	Baja	Ninguna	Bajo
Exportacion PDF	Media	ReportLab o similar	Medio
Estadisticas	Baja	Ninguna	Bajo

10. Priorizacion y Roadmap

La priorizacion se basa en tres ejes: impacto para el usuario (cuanto mejora la experiencia diaria), complejidad de implementacion (cuanto riesgo de regresion), y alineacion con la filosofia VitaZen (honestidad, equilibrio, profundidad sin drama). Las funcionalidades se organizan en tres fases: imprescindibles (mejora inmediata y evidente), recomendadas (alto valor con complejidad moderada), y futuras (requieren mas madurez del sistema).

10.1 Funcionalidades Imprescindibles (FASE 3.1)

Estas funcionalidades deben implementarse en la FASE 3.1 porque proporcionan la mayor mejora en la experiencia diaria con el menor riesgo y coste. Son el minimo viable para que VitaZen compita en calidad con aplicaciones de IA de referencia.

Prioridad	Funcionalidad	Justificacion
P0	Streaming de respuestas	Impacto UX masivo. Sin esto, la app se siente lenta. Groq lo soporta nativamente.
P0	Renderizado Markdown	El Mentor ya genera markdown. Sin renderizado, se pierde formato. Bajo coste.
P0	Copiar mensajes	Funcionalidad mas pedida. Cero riesgo. 1 hora de trabajo.
P1	Input multilinea (textarea)	El cambio es trivial (5 lineas). Impacto inmediato en expresividad.
P1	Regenerar ultima respuesta	Alto valor percibido. Coste de 1 llamada Groq. Sin creditos nuevos.
P2	Busqueda de conversaciones	Esencial con 20+ threads. tsvector es la solucion mas eficiente.
P2	Favoritos + Fijar	Bajo coste, alta utilidad para organizacion personal.

10.2 Funcionalidades Recomendadas (FASE 3.1b)

Estas funcionalidades se recomiendan implementar en la misma FASE 3.1 pero como segundo lote, despues de las imprescindibles. Aportan valor significativo pero tienen mayor complejidad o menor

impacto inmediato.

Prioridad	Funcionalidad	Justificacion
P3	Papelera (soft delete)	Proteccion contra eliminacion accidental. Patron estandar de la industria.
P3	Exportacion TXT/Markdown	Los usuarios invierten reflexion en las conversaciones. Deben poder conservarlas.
P4	Descomposicion de MentorChat	Necesaria para mantener el codigo manejable. Hacer antes de anadir mas funcionalidades.
P4	Estadisticas basicas	Valor motivacional. "Has enviado 47 mensajes esta semana" refuerza el compromiso.

10.3 Funcionalidades Descartadas o Postergadas

Funcionalidad	Veredicto	Razon
Compartir conversaciones	DESCARTAR	Riesgo de privacidad. La exportacion cubre la necesidad.
Exportacion a PDF	POSTERGAR (FASE 3.2)	Coste de generacion elevado. TXT/MD cubren el 90% del uso.
Editar mensajes	POSTERGAR (FASE 3.2)	Alta complejidad y riesgo de confusion. La regeneracion cubre la necesidad de correccion.
Estadisticas avanzadas	POSTERGAR (FASE 3.2)	Requieren queries agregadas y diseño de dashboard.
Adjuntar archivos	DESCARTAR	Fuera del alcance. El Mentor es textual.
Notificaciones de conversaciones	POSTERGAR (FASE 4)	Requiere sistema de notificaciones ya existente pero sin integracion con chat.
Temas/etiquetas en threads	POSTERGAR (FASE 3.2)	Requiere clasificacion automatica. Alta complejidad.
Respuestas multiples (branching)	DESCARTAR	Fuera de la filosofia VitaZen. Una conversacion, un hilo.

11. Funcionalidades Imprescindibles

Tras el analisis completo, se identifican 6 funcionalidades como imprescindibles para que VitaZen alcance un nivel de calidad comparable al de las mejores aplicaciones de IA y productividad. Estas funcionalidades no son "bonitas de tener" sino "el minimo que cualquier chat de IA moderna deberia ofrecer". Sin ellas, la experiencia del usuario es objetivamente inferior a la de ChatGPT, Claude, Gemini, o cualquier otro competidor directo.

- 1. Streaming de respuestas:** Elimina la percepcion de latencia de 2-5 segundos. El usuario ve la respuesta generarse token a token. Groq lo soporta de forma nativa via su SDK. Es el cambio singular con mayor impacto en la calidad percibida. Sin streaming, ninguna otra mejora compensa la sensacion de "espera eterna".
- 2. Renderizado Markdown:** Las respuestas del Mentor ya contienen formato markdown (negritas, listas, bloques de codigo, cabeceras). Sin renderizado, este formato se muestra como texto plano, haciendo que respuestas bien estructuradas se vean como bloques densos e ilegibles. react-markdown (~15KB gzip) resuelve esto sin riesgo de regresiones.
- 3. Copiar mensajes:** Es la funcionalidad mas facil de implementar (1 hora) y la mas inmediatamente visible para el usuario. Sin ella, conservar una respuesta valiosa del Mentor requiere seleccionar texto manualmente, lo cual es tedioso especialmente en movil.
- 4. Input multilinea:** Un campo de texto de una sola linea es inadecuado para un mentor de desarrollo personal. Los usuarios necesitan escribir descripciones contextuales, reflexiones y narrativas que naturalmente ocupan multiples lineas. El cambio de input a textarea es trivial y no tiene riesgo de regresiones.
- 5. Regenerar respuesta:** Cuando el Mentor no acierta, el usuario necesita poder pedir un reintento sin consumir un credito ni anadir ruido al historial. La regeneracion es un patron estandar en todos los chats de IA de referencia.
- 6. Busqueda de conversaciones:** Con 20 threads activos (FREE) y hasta 100 (ELITE), encontrar una conversacion anterior sin busqueda es frustrante. PostgreSQL tsvector con GIN index proporciona busqueda full-text en <5ms sin servicios externos ni coste recurrente.

12. Mejoras Adicionales para Destacar

Más allá de las funcionalidades estándar, VitaZen puede diferenciarse de competidores genéricos (ChatGPT, Claude) aprovechando su ecosistema integrado de bienestar. Ningún chatbot genérico tiene acceso a los datos reales de hábitos, emociones, finanzas y salud del usuario. Las siguientes mejoras aprovechan esta ventaja única.

Contexto visible en la UI: Los motores ya inyectan datos del usuario en el prompt del sistema (emociones, objetivos, hábitos, etapas de vida). Pero esta información es invisible para el usuario. Mostrar un indicador sutil de "El Mentor sabe que estás estresado hoy" o "Objetivo activo: caminar 10.000 pasos" en el header del chat haría tangible la inteligencia del sistema y reforzaría la confianza del usuario en que realmente le está hablando a alguien que le conoce.

Sugerencias contextuales: Los chips de sugerencias son actualmente estáticos (3 de 11, elegidos al azar en el mount). Podrían ser dinámicos basados en el Goals Engine (sugerir "Como va mi objetivo de..."), el Emotional State (sugerir "Hablemos de lo que te tiene estresado") o el patrón de uso reciente (sugerir temas nuevos cuando el usuario repite los mismos).

Resumen de sesión: Al finalizar una conversación (o al cerrar el drawer), mostrar un resumen de 1 línea: "Hablamos sobre estrés laboral (3 mensajes). El Mentor te sugirió una técnica de respiración." Esto proporciona cierre y facilita encontrar conversaciones anteriores.

Conexión con el dashboard: Desde el sidebar del Mentor, mostrar métricas relevantes del día (racha de hábitos, check-in de hoy, estado emocional) que contextualicen la conversación. Esto convierte al Mentor en un punto de convergencia de toda la experiencia VitaZen, no en un módulo aislado.

Atajos de teclado: Enter para enviar, Shift+Enter para nueva línea, Ctrl+Shift+C para copiar último mensaje, Ctrl+Shift+R para regenerar. Los atajos son estándar en chats de IA y su ausencia se nota especialmente en usuarios de escritorio.